RESCIENCE C — OPEN-SOURCE SOFTWARE PAPER

OMR Suite: A Multi-Engine, Privacy-First Solution for Optical Mark Recognition and Assessment

Prof. Dr. Francisco Pérez García[1,2*]

[1] Department of Pharmacology, School of Pharmacy and Food Sciences, University of Barcelona, Spain
[2] Department of Technology, Pompeu Fabra High School, Martorell (Barcelona), Spain
[*] Corresponding author: perez@ub.edu | ORCID: 0000-0002-1395-5573

Date: July 2026 | Repository: https://github.com/drfperez/OMRSuite

Summary

Educational institutions and researchers frequently require reliable, cost-effective methods for evaluating multiple-choice exams. `OMR Suite` is an open-source, privacy-first software system designed to generate, personalize, and automatically grade Optical Mark Recognition (OMR) answer sheets.

Unlike traditional OMR solutions that rely on specialized scanning hardware or cloud backend servers, `OMR Suite` provides a unified computer vision pipeline implemented across three distinct processing architectures:

1. A **100% client-side HTML5/WebAssembly engine** (`OpenCV.js` and `jsPDF`) running entirely within modern web browsers without server interaction.
2. A **Python cloud/batch engine** (`OpenCV` and `Pandas`) optimized for heavy bulk workloads inside Jupyter environments or Google Colab.
3. A **Native C++ engine** (`OpenCV 4`) compiled to a standalone executable for high-throughput, low-latency desktop execution.

All three engines share a standardized geometry model and a 4-stage computer vision pipeline capable of automatically correcting page rotation, perspective distortion, and illumination variance while evaluating up to 150 questions per sheet.

Statement of Need

While commercial OMR systems and cloud-based grading platforms exist, they present significant barriers for educators and academic researchers:

- **Privacy and Compliance:** Regulations such as GDPR strictly limit the transmission of identifiable student records to third-party cloud backends.
- **Infrastructure and Cost:** Specialized hardware scanners and proprietary software licenses impose non-trivial financial burdens on lower-resource academic institutions.
- **Resource Limits in Browser Environments:** Client-side web implementations leveraging WebAssembly provide ideal zero-installation workflows; however, web browsers enforce rigid memory allocation caps per tab. Grading hundreds of high-resolution images in a single web tab can trigger out-of-memory crashes.

`OMR Suite` directly addresses these limitations by pairing a serverless, privacy-preserving web application with fallback Python and C++ engines. Teachers can generate and grade exams in browser mode for daily classroom use, or seamlessly pivot to the Python/C++ binaries for large-scale institutional evaluation without changing answer sheet templates.

State of the Field

Several open-source OMR packages currently exist in the academic and educational ecosystem:

- **Auto-Multiple-Choice (AMC)** (Bienvenue, 2021): A mature, highly flexible OMR system based on LaTeX. While powerful, AMC requires a complex local LaTeX installation, operates primarily on Unix-like platforms, and lacks a zero-installation browser interface.
- **SDAPS** (Waechter, 2019): A Python-based OMR utility focused on survey parsing. It relies heavily on specific backend dependencies and lacks real-time, client-side browser evaluation.
- **FormScanner** (Giammettei, 2017): A desktop Java application for reading OMR forms. It requires local runtime installation (JRE) and lacks multi-engine cloud or web integrations.

`OMR Suite` fills a distinct gap by combining **zero-setup browser execution**, **native C++ desktop execution**, and **cloud-ready Python notebooks** around a single standardized 150-question A4 template format.

Software Design

The core system architecture consists of a unified computer vision pipeline implemented across JavaScript, Python, and C++.

```
[ Input Exam Scan ] → [ 1. Grayscale & Binarization ] → [ 2. 4-Corner Fiducial Detection ]
                                                                ↓
[ Output: CSV Grade Report ] ← [ 4. ROI Pixel Density Sampling ] ← [ 3. Perspective Warp (1050x1485 Grid) ]
```

Computer Vision Pipeline Specifications

1. **Preprocessing and Binarization:** Scanned exam sheets are converted to grayscale and thresholded using inverse binarization:

$$I_{thresh}(x,y) = 255 \text{ if } I(x,y) < T, \text{ else } 0 \quad (T = 120)$$

2. **Fiducial Registration:** The algorithm extracts external contours and filters candidates by area A ($200 < A < 30000$), aspect ratio ($0.7 \le w/h \le 1.3$), and convexity ratio ($A / A_{hull} > 0.8$). The centers of mass (C_x, C_y) are computed via spatial moments:

$$C_x = M_{10} / M_{00}, \quad C_y = M_{01} / M_{00}$$

3. **Homography and Perspective Normalization:** The detected corners are mapped to predefined fixed coordinates on a canonical 1050 × 1485 pixel template grid using a perspective transformation matrix $M_{perspective}$, removing page skew, tilt, and scaling artifacts.
4. **Region of Interest (ROI) Evaluation:** Bubbles are evaluated by sampling 10 × 10 pixel bounding boxes at normalized coordinates (x_i, y_i). A bubble is flagged as filled if the ratio of active foreground pixels exceeds 40%:

$$\text{Fill Ratio} = (1 / (W \cdot H)) \sum \sum I(I_{thresh}(x,y) == 255) > 0.40$$

Research and Educational Impact

`OMR Suite` provides an accessible, zero-cost assessment platform for schools, universities, and educational researchers. By keeping all computations local within the browser or via standalone native executables, the software guarantees compliance with strict data protection regulations.

The software has demonstrated high grading accuracy across various image resolutions, lighting conditions, and camera angles. Its open-source codebase allows researchers to extend the vision algorithms, adapt answer sheet layouts, or integrate the core C++/Python modules into larger learning management systems (LMS) and automated research pipelines.

AI Usage Disclosure

No generative AI tools were used in the core computer vision algorithm development or primary software architecture of this package.

Acknowledgements

The author acknowledges the open-source maintainers of OpenCV (Bradski, 2000), OpenCV.js, and jsPDF (2021), whose foundational libraries enabled the multi-platform architecture of this software.

References

Bienvenue, A. (2021). *Auto-Multiple-Choice: Multiple choice papers management*. https://www.bodes.org/auto-multiple-choice/

Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*, 25(11), 120–123.

Giammettei, A. (2017). *FormScanner: OMR Software for processing multiple choice forms*. SourceForge.

Haas, A., Rossberg, A., Schuff, D. L., Titzer, B. L., Holman, M., Gohman, D., Wagner, L., Zakai, A., & Bastien, J. (2017). Bringing the web up to speed with WebAssembly. *ACM SIGPLAN Notices*, 52(6), 185–200.

Parallaxes et al. (2021). *jsPDF: A HTML5 client-side solution for generating PDFs*. GitHub repository.

Waechter, B. (2019). *SDAPS: Scripts for data collection and processing*. SDAPS Project.